rqbit 인증 및 보안 기능 분석

개요

이 문서는 rqbit BitTorrent 클라이언트의 HTTP API에서 제공하는 인증 및 보안 기능을 분석합니다. rqbit은 HTTP 기본 인증, CORS 정책, IP 차단 목록, 속도 제한 등 다양한 보안 메커니즘을 구현하여 안전한 API 접근을 보장합니다.

HTTP 기본 인증 구현

1. 인증 설정 구조

HttpApiOptions 구조체

```rust
#[derive(Debug, Default)]
pub struct HttpApiOptions {
pub read_only: bool,
pub basic_auth: Option<(String, String)>,
pub allow_create: bool,
#[cfg(feature = "prometheus")]
pub prometheus_handle: Option<metrics_exporter_prometheus::PrometheusHandle>,
}
```

특징:

- **선택적 인증**: `basic_auth` 가 `None` 이면 인증 없이 접근 가능

- **단순한 구조**: 사용자명과 비밀번호의 튜플로 구성
- **읽기 전용 모드**: 인증과 별개로 쓰기 작업 제한 가능

2. 기본 인증 미들웨어

simple_basic_auth 함수

```
async fn simple_basic_auth(
expected_username: Option<&str>,
expected_password: Option<&str>,
headers: HeaderMap,
request: axum::extract::Request,
next: Next,
) -> Result<axum::response::Response> {
let (expected_user, expected_pass) = match (expected_username, expected_password
(Some(u), Some(p)) => (u, p),
_ => return Ok(next.run(request).await),
};
let user_pass = headers
.get("Authorization")
.and_then(|h| h.to_str().ok())
.and_then(|h| h.strip_prefix("Basic "))
.and_then(|v| base64::engine::general_purpose::STANDARD.decode(v).ok())
.and_then(|v| String::from_utf8(v).ok());
let user_pass = match user_pass {
```

```
Some(user_pass) => user_pass,

None => {

return Ok((

StatusCode::UNAUTHORIZED,

[("WWW-Authenticate", "Basic realm=\"API\"")],

).into_response());

}

};

// TODO: constant time compare

match user_pass.split_once(':') {

Some((u, p)) if u == expected_user && p == expected_pass => Ok(next.run(request)

_ => Err(ApiError::unathorized()),

}

}
```

특징:

- **표준 HTTP Basic Auth**: RFC 7617 준수
- **Base64 디코딩**: Authorization 헤더의 Base64 인코딩 처리
- **WWW-Authenticate 헤더**: 인증 실패 시 브라우저 인증 프롬프트 유도
- **조기 반환**: 인증이 설정되지 않은 경우 즉시 통과
- **보안 개선 여지**: TODO 주석에서 상수 시간 비교 필요성 언급

3. 미들웨어 적용

라우터 레벨 인증

```
// Simple one-user basic auth
if let Some((user, pass)) = state.opts.basic_auth.clone() {
info!("Enabling simple basic authentication in HTTP API");
main_router = main_router.route_layer(axum::middleware::from_fn(
move |headers, request, next| {
let user = user.clone();
let pass = pass.clone();
async move {
simple_basic_auth(Some(&user), Some(&pass), headers, request, next).await
}
},
));
}
```

특징:

- **전역 적용**: 모든 API 엔드포인트에 인증 적용
- **클로저 캡처**: 사용자 자격 증명을 클로저로 캡처
- **조건부 활성화**: 인증 정보가 제공된 경우에만 활성화

CORS 설정 및 보안 헤더 관리

1. CORS 정책 구현

허용된 오리진 설정

```
let cors_layer = {
use tower_http::cors::{AllowHeaders, AllowOrigin};

const ALLOWED_ORIGINS: [&[u8]; 4] = [
// Webui-dev
b"http://localhost:3031",
b"http://127.0.0.1:3031",
// Tauri dev
b"http://localhost:1420",
// Tauri prod
b"tauri://localhost",
];

let allow_regex = std::env::var("CORS_ALLOW_REGEXP")
.ok()
.and_then(|value| regex::bytes::Regex::new(&value).ok());

tower_http::cors::CorsLayer::default()
.allow_origin(AllowOrigin::predicate(move |v, _| {
ALLOWED_ORIGINS.contains(&v.as_bytes())
|| allow_regex
```

```
.as_ref()
.map(move |r| r.is_match(v.as_bytes()))
.unwrap_or(false)
}))
.allow_headers(AllowHeaders::any())
};
```

특징:

- **화이트리스트 방식**: 명시적으로 허용된 오리진만 접근 가능
- **개발 환경 지원**: 로컬 개발 서버와 Tauri 앱 지원
- **동적 설정**: 환경 변수를 통한 추가 오리진 허용
- **정규식 지원**: 복잡한 오리진 패턴 매칭 가능
- **유연한 헤더**: 모든 헤더 허용으로 개발 편의성 제공

2. 보안 헤더 설정

트레이싱 미들웨어

```
.layer(
tower_http::trace::TraceLayer::new_for_http()
.make_span_with(|req: &Request| {
let method = req.method();
let uri = req.uri();
if let Some(ConnectInfo(addr)) = req
.extensions()
```

```
.get::<ConnectInfo<librqbit_dualstack_sockets::WrappedSocketAddr>>()
{
debug_span!("request", %method, %uri, addr=%addr.0)
} else {
debug_span!("request", %method, %uri)
}
})
.on_request(|req: &Request, _: &Span| {
if req.uri().path().starts_with("/upnp") {
debug!(headers=?req.headers())
}
})
.on_response(DefaultOnResponse::new().include_headers(true))
.on_failure(/* ... */)
)
```

특징:

- **요청 추적**: 모든 HTTP 요청에 대한 상세 로깅
- **클라이언트 IP 기록**: 연결 정보를 통한 클라이언트 식별
- **헤더 로깅**: UPnP 요청에 대한 상세 헤더 정보 기록
- **응답 추적**: 응답 시간과 상태 코드 모니터링

API 접근 제어 및 권한 관리

1. 읽기 전용 모드

조건부 엔드포인트 등록

```
if !state.opts.read_only {
api_router = api_router
.route("/torrents", post(torrents::h_torrents_post))
.route("/torrents/limits", post(configure::h_update_session_ratelimits))
.route("/torrents/{id}/pause", post(torrents::h_torrent_action_pause))
.route("/torrents/{id}/start", post(torrents::h_torrent_action_start))
.route("/torrents/{id}/forget", post(torrents::h_torrent_action_forget))
.route("/torrents/{id}/delete", post(torrents::h_torrent_action_delete))
.route("/torrents/{id}/update_only_files", post(torrents::h_torrent_action_updat
.route("/torrents/{id}/add_peers", post(torrents::h_add_peers))
.route("/torrents/create", post(torrents::h_create_torrent));
}
```

특징:

- **컴파일 타임 제어**: 읽기 전용 모드에서는 쓰기 엔드포인트가 라우터에 등록되지 않음
- **404 응답**: 비활성화된 엔드포인트 접근 시 자동으로 404 반환
- **세밀한 제어**: 개별 작업별로 접근 제어 가능

2. 오류 처리 및 상태 코드

ApiError 구조체

```rust
#[derive(Debug)]
pub struct ApiError {
status: Option<StatusCode>,
kind: ApiErrorKind,
}

#[derive(thiserror::Error, Debug)]
pub enum ApiErrorKind {
#[error("torrent not found {0}")]
TorrentNotFound(TorrentIdOrHash),
#[error("DHT is disabled")]
DhtDisabled,
#[error("unauthorized")]
Unauthorized,
#[error("{0}")]
Text(&'static str),
#[error(transparent)]
OtherAnyhow(#[from] anyhow::Error),
// ... 기타 오류 타입들
}
```

특징:

- **구조화된 오류**: 오류 종류별로 명확한 분류

- **HTTP 상태 코드**: 각 오류에 적절한 HTTP 상태 코드 매핑
- **사용자 친화적 메시지**: 기술적 세부사항보다는 이해하기 쉬운 메시지

인증 오류 처리

```
pub const fn unathorized() -> Self {
Self {
status: Some(StatusCode::UNAUTHORIZED),
kind: ApiErrorKind::Unauthorized,
}
}
```

특징:

- **표준 상태 코드**: HTTP 401 Unauthorized 사용
- **일관된 응답**: 모든 인증 실패에 대해 동일한 응답 형식

IP 차단 목록 (Blocklist) 시스템

1. Blocklist 구조체

효율적인 IP 범위 저장

```
pub struct Blocklist {
// We could store only one interval tree, but splitting them takes less memory,
// as IpAddr is 17 bytes, Ipv4Addr is only 4 bytes (the majority of ranges).
v4: IntervalTreeWithSize<Ipv4Addr>,
```

```
v6: IntervalTreeWithSize<Ipv6Addr>,
}

struct IntervalTreeWithSize<T> {
t: IntervalTree<T, ()>,
len: usize,
}
```

특징:

- **메모리 최적화**: IPv4와 IPv6를 분리하여 메모리 사용량 최소화
- **인터벌 트리**: 효율적인 범위 검색을 위한 자료구조 사용
- **크기 추적**: 차단 규칙 수를 별도로 추적

2. 차단 목록 로딩

다양한 소스 지원

```
pub async fn load_from_url(url: &str) -> Result<Self> {
let parsed_url = Url::parse(url).context("failed to parse URL")?;

if parsed_url.scheme() == "file" {
let path = parsed_url
.to_file_path()
.ok()
.context("failed to convert file URL to path")?;
```

```
return Self::load_from_file(path).await;
}

let response = reqwest::get(parsed_url)
.await
.context("error fetching blocklist")?;
if !response.status().is_success() {
anyhow::bail!("error fetching blocklist: HTTP {}", response.status());
}

let mut reader = StreamReader::new(response.bytes_stream().map_err(std::io::Erro
let bl = Self::create_from_stream(&mut reader).await?;
Ok(bl)
}
```

특징:

- **URL 지원**: HTTP/HTTPS URL에서 차단 목록 다운로드
- **파일 지원**: 로컬 파일에서 차단 목록 로드
- **스트림 처리**: 대용량 파일의 효율적 처리
- **오류 처리**: 네트워크 오류와 파일 오류에 대한 적절한 처리

압축 파일 지원

```
async fn create_from_stream(reader: &mut (dyn AsyncRead + Unpin + Send)) -> Resu
```

```
let mut peek_bytes = [0u8; 2];

let mut reader = tokio::io::BufReader::new(reader);



// Peek the first bytes by filling buffer

let buffer = reader.fill_buf().await?;

if buffer.len() >= 2 {

peek_bytes.copy_from_slice(&buffer[0..2]);

} else {

anyhow::bail!("content too short: not enough data to determine compression");

}



// Check for Gzip magic bytes

let is_gzip = peek_bytes == [0x1F, 0x8B];



if is_gzip {

trace!("detected gzip stream, decompressing");

Self::create_from_decoded_stream(&mut BufReader::new(GzipDecoder::new(reader))).

} else {

trace!("plain text file detected.");

Self::create_from_decoded_stream(&mut reader).await

}

}
```

특징:

- **자동 압축 감지**: Gzip 매직 바이트를 통한 자동 감지
- **투명한 압축 해제**: 사용자가 압축 여부를 신경 쓸 필요 없음
- **메모리 효율성**: 스트림 기반 압축 해제

3. IP 차단 검사

효율적인 검색

```
pub fn is_blocked(&self, ip: IpAddr) -> bool {
match ip {
IpAddr::V4(a) => self.v4.t.query_point(a).next().is_some(),
IpAddr::V6(a) => self.v6.t.query_point(a).next().is_some(),
}
}
```

특징:

- **O(log n) 검색**: 인터벌 트리를 통한 효율적 검색
- **타입 안전성**: IPv4와 IPv6를 별도로 처리
- **단순한 인터페이스**: 불린 값으로 간단한 결과 반환

차단 목록 형식 파싱

```
fn parse_ip_range(line: &str) -> Option<IpRange> {
let line = line.trim();
if line.starts_with('#') || line.is_empty() {
```

```
return None;
}


let (_name, ips) = {
let is_ipv6 = line.matches(":").count() > 2;
if is_ipv6 {
line.split_once(':')?
} else {
line.rsplit_once(':')?
}
};
let (start, end) = ips.split_once('-')?;
match (IpAddr::from_str(start).ok()?, IpAddr::from_str(end).ok()?) {
(IpAddr::V4(start), IpAddr::V4(end)) => {
let end = Ipv4Addr::from_bits(end.to_bits().saturating_add(1));
Some(IpRange::V4(start..end))
}
(IpAddr::V6(start), IpAddr::V6(end)) => {
let end = Ipv6Addr::from_bits(end.to_bits().saturating_add(1));
Some(IpRange::V6(start..end))
}
_ => None,
}
```

```
}
```

특징:

- **유연한 형식**: `name:start-end` 형식 지원
- **주석 지원**: `#` 으로 시작하는 주석 라인 무시
- **IPv4/IPv6 자동 감지**: 콜론 개수로 IP 버전 판별
- **범위 처리**: 시작-끝 IP 범위를 Rust Range로 변환

4. 실제 차단 적용

피어 연결 시 차단 검사

```
let is_in_blocklist = session.blocklist.is_blocked(addr.ip());
if is_in_blocklist {
session
.stats
.counters
.blocked_outgoing
.fetch_add(1, std::sync::atomic::Ordering::Relaxed);
return Err(Error::PeerBlocked);
}
```

특징:

- **연결 전 검사**: 실제 네트워크 연결 전에 차단 여부 확인
- **통계 수집**: 차단된 연결 시도 횟수 추적

- **조기 반환**: 차단된 IP에 대해서는 즉시 연결 시도 중단

속도 제한 (Rate Limiting) 시스템

1. Limits 구조체

업로드/다운로드 속도 제한

```
#[derive(Default, Serialize, Deserialize, Clone, Copy, Debug, PartialEq, Eq)]
pub struct LimitsConfig {
pub upload_bps: Option<NonZeroU32>,
pub download_bps: Option<NonZeroU32>,
}

pub struct Limits {
down: Limit,
up: Limit,
}

struct Limit(ArcSwapOption<RateLimiter>);
```

특징:

- **분리된 제한**: 업로드와 다운로드 속도를 독립적으로 제한
- **선택적 제한**: `None` 으로 설정하여 제한 비활성화 가능
- **동적 변경**: `ArcSwapOption` 을 통한 런타임 제한 변경

- **스레드 안전성**: 여러 스레드에서 동시 접근 가능

2. 토큰 버킷 알고리즘

Governor 크레이트 활용

```
impl Limit {
fn new_inner(bps: Option<NonZeroU32>) -> Option<Arc<RateLimiter>> {
let bps = bps?;
Some(Arc::new(RateLimiter::direct(Quota::per_second(bps))))
}

async fn acquire(&self, size: NonZeroU32) -> crate::Result<()> {
let lim = self.0.load().clone();
if let Some(rl) = lim.as_ref() {
rl.until_n_ready(size).await?;
}
Ok(())
}
}
```

특징:

- **토큰 버킷**: Governor 크레이트의 토큰 버킷 알고리즘 사용
- **초당 할당량**: 초당 바이트 수로 속도 제한 설정
- **비동기 대기**: 토큰이 부족할 때 비동기적으로 대기

- **정확한 제어**: 요청 크기에 비례한 토큰 소비

3. 계층적 속도 제한

세션 레벨과 토렌트 레벨

```
// 토렌트 레벨 제한
self.ratelimits
.prepare_for_upload(NonZeroU32::new(ci.size).unwrap())
.await?;

// 세션 레벨 제한
if let Some(session) = self.shared.session.upgrade() {
session
.ratelimits
.prepare_for_upload(NonZeroU32::new(ci.size).unwrap())
.await?;
}
```

특징:

- **이중 제한**: 개별 토렌트와 전역 세션 모두에서 속도 제한
- **순차 적용**: 토렌트 제한을 먼저 통과한 후 세션 제한 적용
- **공정한 분배**: 여러 토렌트 간 대역폭의 공정한 분배

4. 동적 제한 변경

런타임 설정 변경

```rust
pub fn set_upload_bps(&self, bps: Option<NonZeroU32>) {
self.up.set(bps);
}

pub fn set_download_bps(&self, bps: Option<NonZeroU32>) {
self.down.set(bps);
}

fn set(&self, limit: Option<NonZeroU32>) {
let new = Self::new_inner(limit);
self.0.swap(new);
}
```

특징:

- **무중단 변경**: 서비스 중단 없이 속도 제한 변경
- **원자적 교체**: `ArcSwapOption` 을 통한 원자적 레이트 리미터 교체
- **즉시 적용**: 새로운 제한이 즉시 모든 연결에 적용

추가 보안 고려사항

1. 입력 검증

토렌트 파일 검증

```rust
fn read_and_validate_torrent(path: &Path) -> anyhow::Result<AddTorrent<'static>>
```

```
let mut buf = Vec::new();

std::fs::File::open(path)

.context("error opening torrent file")?

.read_to_end(&mut buf)

.context("error reading torrent file")?;

// 토렌트 파일 형식 검증 로직...

Ok(AddTorrent::TorrentFileBytes(buf.into()))

}
```

특징:

- **파일 형식 검증**: 토렌트 파일의 유효성 검사
- **크기 제한**: 메모리 사용량 제한을 통한 DoS 방지
- **오류 처리**: 잘못된 파일에 대한 적절한 오류 메시지

2. 리소스 제한

동시 연결 수 제한

```
// Limits how many active (occupying network resources) peers there are at a mom

peer_semaphore: Arc<Semaphore>,


// 연결 시도 시

let permit = match self.state.peer_semaphore.try_acquire() {

Ok(permit) => permit,

Err(_) => {
```

```
debug!("limit of live peers reached, dropping incoming peer");
// 연결 거부 로직...
}
};
```

특징:

- **세마포어 기반**: Tokio 세마포어를 통한 동시 연결 수 제한
- **리소스 보호**: 과도한 연결로 인한 시스템 리소스 고갈 방지
- **우아한 거부**: 제한 초과 시 적절한 로깅과 함께 연결 거부

3. 로깅 및 모니터링

보안 이벤트 로깅

```
if is_in_blocklist {
debug!("peer {} is in blocklist, not connecting", addr);
session
.stats
.counters
.blocked_outgoing
.fetch_add(1, std::sync::atomic::Ordering::Relaxed);
}
```

특징:

- **상세한 로깅**: 보안 관련 이벤트의 상세한 로그 기록

- **통계 수집**: 차단된 연결 시도 등의 통계 정보 수집
- **모니터링 지원**: Prometheus 메트릭을 통한 모니터링 가능

보안 개선 여지

1. 현재 제한사항

인증 시스템

- **단일 사용자**: 현재는 하나의 사용자 계정만 지원
- **상수 시간 비교**: 타이밍 공격 방지를 위한 상수 시간 비교 미구현
- **세션 관리**: 세션 기반 인증이나 토큰 기반 인증 미지원

암호화

- **HTTPS 미지원**: 현재는 HTTP만 지원하여 통신 내용이 평문으로 전송
- **인증서 관리**: TLS 인증서 자동 관리 기능 없음

2. 권장 개선사항

인증 강화

```
// 개선 예시: 상수 시간 비교
use subtle::ConstantTimeEq;

fn secure_compare(a: &str, b: &str) -> bool {
if a.len() != b.len() {
return false;
}
```

```
a.as_bytes().ct_eq(b.as_bytes()).into()

}
```

HTTPS 지원

- **TLS 터미네이션**: 리버스 프록시를 통한 HTTPS 지원
- **자체 서명 인증서**: 개발 환경을 위한 자체 서명 인증서 생성
- **Let's Encrypt 통합**: 자동 인증서 발급 및 갱신

결론

rqbit의 보안 시스템은 다음과 같은 특징을 보여줍니다:

1. **다층 보안**: 인증, CORS, IP 차단, 속도 제한 등 다양한 보안 계층
2. **실용적 접근**: 복잡하지 않으면서도 효과적인 보안 메커니즘
3. **성능 고려**: 보안 기능이 성능에 미치는 영향 최소화
4. **유연한 설정**: 환경에 따른 보안 정책 조정 가능
5. **모니터링 지원**: 보안 이벤트 로깅과 통계 수집
6. **개선 여지**: 더 강화된 보안을 위한 명확한 개선 방향 제시

이러한 설계는 BitTorrent 클라이언트의 특성을 고려하면서도 웹 API의 보안 요구사항을 적절히 만족시키는 균형잡힌 접근법을 보여줍니다.